

Collaborative Sentiment Analysis Project Report

Team Members : Melek, Amisha, Ariel

Github repository : https://github.com/AmuuLakh/MLOps_Pipeline

1. Overview of the Chosen Approach

Our project focuses on building an **end-to-end sentiment analysis pipeline using a BERT-based model**, following an **MLOps-oriented approach** to ensure modularity, collaboration, and reproducibility.

The workflow was **inspired by the Kaggle notebook** "[Sentiment Analysis using BERT](#)" by Prakhar Rathi.

We adapted its structure and logic but refactored it into independent, testable Python modules, integrated with GitHub, Trello, and continuous integration (CI).

The complete pipeline included:

1. **Data Extraction** – loading and validating raw text data.
2. **Data Cleaning & Normalization** – text preprocessing (lowercasing, removing punctuation, handling emojis, contractions).
3. **Tokenization** – splitting text into tokens using Hugging Face's `AutoTokenizer`.
4. **Model Fine-Tuning** – fine-tuning `bert-base-uncased` on a sentiment dataset.
5. **Inference** – building a function to predict sentiment on new inputs.
6. **Testing & Logging** – automated verification with `pytest` and dedicated log files.

The modular structure followed this layout:

```
project/
├─ requirements.txt
├─ data_cleaning.log
├─ data_log.log
├─ model_training.log
├─ src/
│   ├─ data_extraction.py
│   ├─ data_processing.py
│   ├─ model.py
│   ├─ inference.py
│   └─ data/
│       ├─ processed/
│       │   ├─ eval_tokenized.csv
│       │   └─ train_tokenized.csv
│       └─ dataset.csv
└─ tests/
    └─ unit/
        ├─ test_data_extraction.py
        ├─ test_data_processing.py
        ├─ test_model.py
        └─ test_inference.py
```

This approach allowed each team member to work independently on separate modules while maintaining a coherent development flow through pull requests and reviews.

2. Division of Labor

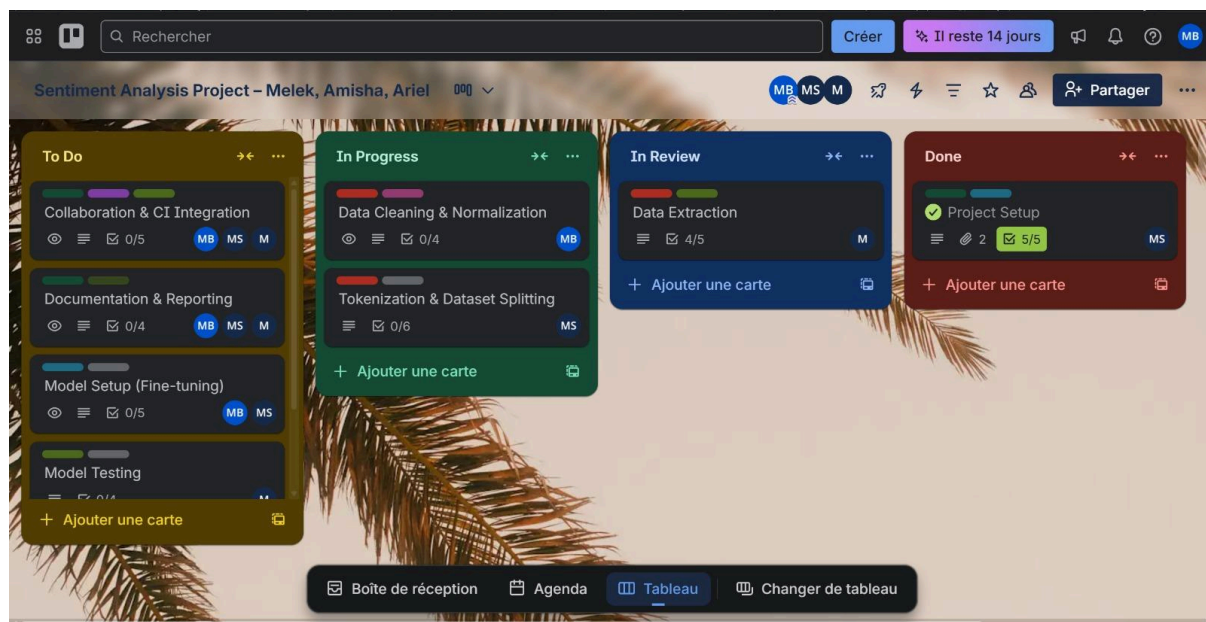
Student	Responsibilities
Amisha	Implemented <code>data_extraction.py</code> , unit tests, and logging. Assisted with debugging failed tests and documenting the project. Implemented <code>model_test.py</code> and <code>inference_test.py</code> .
Melek	Collaborated on <code>data_processing.py</code> (text cleaning & normalization). Wrote preprocessing tests for the implemented part. Implemented <code>model.py</code> to train and save the model. Added the <code>inference.py</code> to predict on test text.

Ariel	Created the initial GitHub repository and prepared the project's structure. Collaborated on <code>data_processing.py</code> (Tokenization & Dataset Splitting). Wrote preprocessing tests for the implemented part. Worked on the reporting and taking screenshots.
--------------	---

Collaboration was achieved via:

- **GitHub** (branching, pull requests, and code reviews).
- **Trello** (for progress tracking).
- **WhatsApp** (for real-time communication).

3. Trello Board Screenshots



Board Structure:

- **To Do:** Tasks pending implementation.
- **In Progress:** Ongoing work with assignees.
- **In Review:** Tasks waiting for PR approval.
- **Done:** Completed and verified components.

Example cards:

Project Setup

+ Ajouter 🕒 Dates 📋 Checklist 📎 Pièce jointe

Membres Étiquettes

MS + documentation backend +

Description Modifier

Initialize the GitHub repository, structure folders (`data_extraction.py` , `data_processing.py` , `model.py` , `inference.py` , `tests/`), and configure the development environment.

Pièces jointes Ajouter

Liens

🔗 Github Repo Link Connectez votre compte Github ...

Fichiers

Folder structure Ajout : 23 oct. 2025, 10:28

📋 Checklist Masquer les tâches cochées Supprimer

100%

- ✓ Create repo and set up .gitignore
- ✓ Add requirements.txt
- ✓ Create main project folder structure
- ✓ Initialize [README.md](#)
- ✓ Set up virtual environment

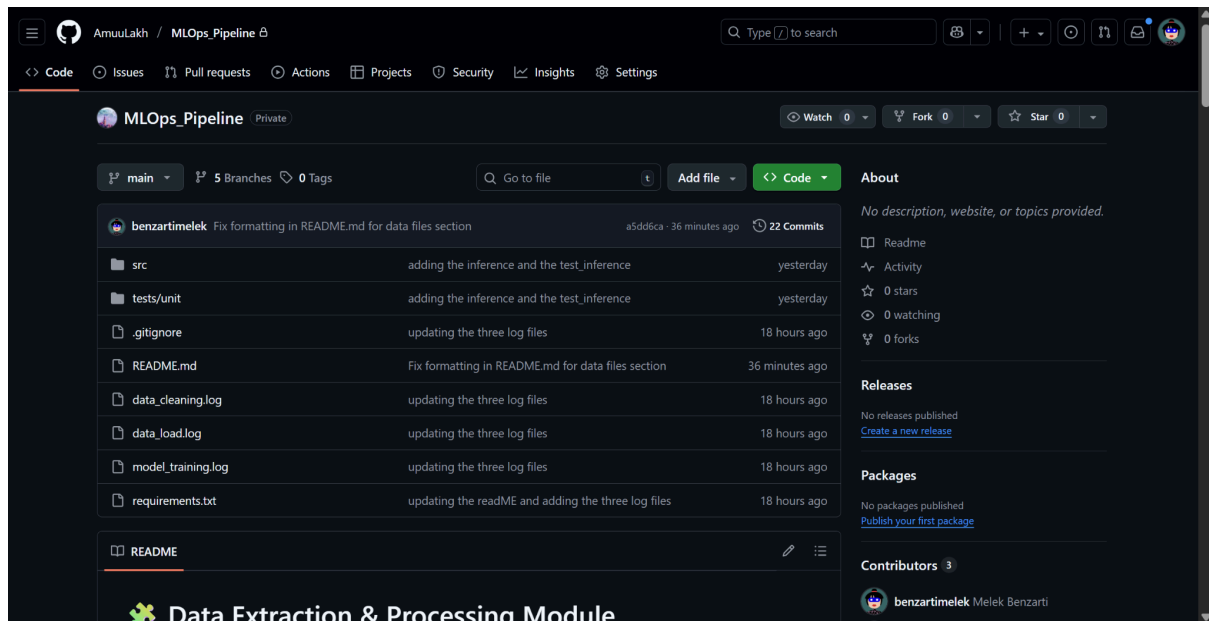
Ajouter un élément

Each card included:

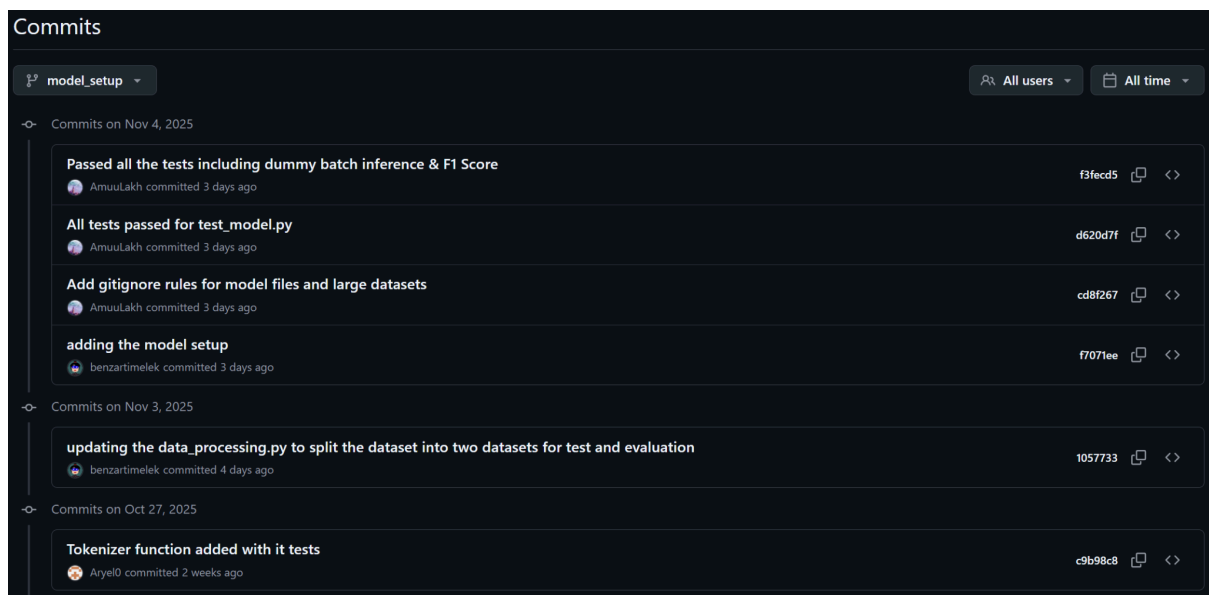
- **Description** of the task.
 - **Checklist** with acceptance criteria.
 - **Labels** (e.g., `data`, `model`, `testing`, `documentation`).
 - **Attachments** (code snippets, GitHub PR links, test results).
-

4. GitHub Screenshots of Commits and Pull Requests

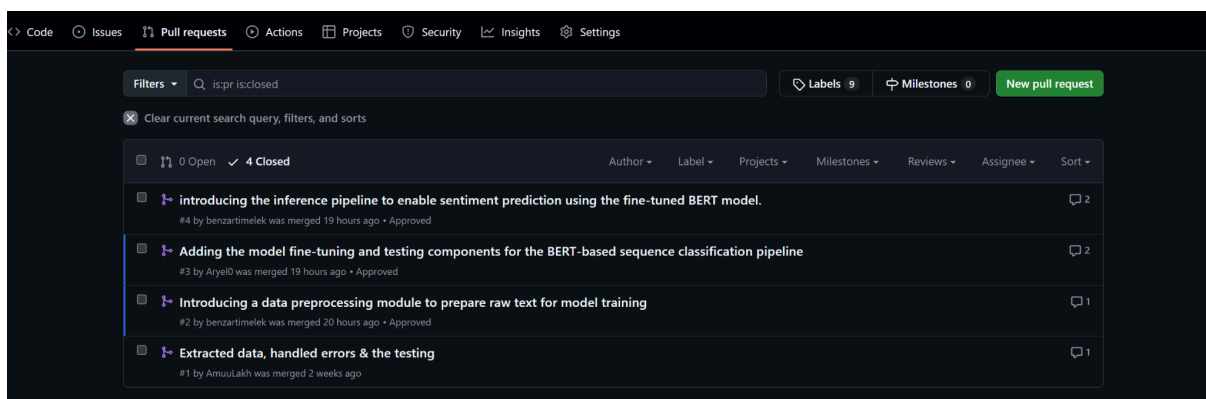
Github Repository (main branch)



Example of commits to the model_setup branch:



Pull requests:



Example of one pull requests with reviews:

Adding the model fine-tuning and testing components for the BERT-based sequence classification pipeline #3

Merged Aryel0 merged 4 commits into `main` from `model_setup` 20 hours ago

Conversation 2 | Commits 4 | Checks 0 | Files changed 3 | +609 -1

Key Updates

- Model Setup (Fine-tuning)**
 - Loaded `AutoModelForSequenceClassification` with `BERT-base-uncased`.
 - Configured optimizer, loss function, and training loop (or Hugging Face Trainer API).
 - Defined evaluation metrics: accuracy and F1-score.
 - Trained the model on the processed dataset.
 - Saved fine-tuned model weights for reuse and deployment.
- Model Testing**
 - Added unit tests under `tests/unit/test_model.py`.
 - Validated model output shape (logits) through dummy batch inference.
 - Verified performance by evaluating accuracy and F1-score on the validation set.

Reviewers

- AmuuLakh ✓
- benzartimelek ✓

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

Notifications Customize

Unsubscribe

You're receiving notifications because your review was requested.

3 participants

Lock conversation

benzartimelek and others added 4 commits 3 days ago

- adding the model setup `f7071ee`
- Add gitignore rules for model files and large datasets `cd8f267`
- All tests passed for `test_model.py` `d628d7f`
- Passed all the tests including dummy batch inference & F1 Score `f3fecd5`

Aryel0 requested review from AmuuLakh and benzartimelek 20 hours ago

benzartimelek approved these changes 20 hours ago [View reviewed changes](#)

benzartimelek left a comment

All good from my side. Good Job

AmuuLakh approved these changes 20 hours ago [View reviewed changes](#)

AmuuLakh left a comment

I went through all the updates — everything looks good!

Aryel0 merged commit `471b362` into `main` 20 hours ago [Revert](#)

Pull request successfully merged and closed

You're all set — the `model_setup` branch can be safely deleted. [Delete branch](#)

GitHub Collaboration Workflow:

- Each member worked on a **feature branch** (`data_extraction`, `model`, etc.).

- Code was integrated through **pull requests** reviewed by teammates.
 - We maintained **clear commit messages**.
-

5. Challenges Faced

Despite following a structured pipeline, several practical challenges emerged during the project:

1. Long Training Time

- Fine-tuning the BERT model on our sentiment dataset took **nearly 5 hours**, even after:
 - Reducing the number of epochs.
 - Lowering batch size.
 - Optimizing GPU utilization.

⇒ Switched to a **DistilBERT** model (smaller model).

2. Persistent Test Failure

- One of our inference tests consistently failed.
- After multiple debugging attempts and code adjustments, we discovered that the issue stemmed from an **incompletely trained model**.
- Since retraining would again require 5 hours, we documented this limitation and decided **not to retrain** within the current project timeline.

```
===== test session starts =====
platform win32 -- Python 3.12.7, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\benza\Desktop\PGE3\MLOPS\project\MLO
ps_Pipeline\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\benza\Desktop\PGE3\MLOPS\project\MLOps_Pipeline
plugins: anyio-4.11.0
collected 32 items

tests/unit/test_data_extraction.py::test_missing_file_returns_none PASSED [ 3%]
tests/unit/test_data_extraction.py::test_load_valid_csv PASSED [ 6%]
tests/unit/test_data_extraction.py::test_empty_file_returns_none PASSED [ 9%]
tests/unit/test_data_extraction.py::test_semicolon_delimiter_detection PASSED [ 12%]
tests/unit/test_data_extraction.py::test_encoding_fallback PASSED [ 15%]
tests/unit/test_data_extraction.py::test_parser_error_fallback PASSED [ 18%]
tests/unit/test_data_processing.py::test_clean_text_with_real_data PASSED [ 21%]
tests/unit/test_data_processing.py::test_normalize_reviews_creates_clean_column PASSED [ 25%]
tests/unit/test_data_processing.py::test_normalize_reviews_no_content_column PASSED [ 28%]
tests/unit/test_data_processing.py::test_tokenization_adds_columns PASSED [ 31%]
tests/unit/test_data_processing.py::test_tokenization_padding_and_mask PASSED [ 34%]
tests/unit/test_data_processing.py::test_tokenization_missing_clean_content PASSED [ 37%]
tests/unit/test_inference.py::test_model_load_and_predict PASSED [ 40%]
tests/unit/test_inference.py::test_positive_sentiment PASSED [ 43%]
tests/unit/test_inference.py::test_negative_sentiment PASSED [ 46%]
tests/unit/test_inference.py::test_neutral_sentiment FAILED [ 50%]
tests/unit/test_inference.py::test_empty_input PASSED [ 53%]
tests/unit/test_inference.py::test_prediction_consistency PASSED [ 56%]
tests/unit/test_model.py::TestModelInstantiation::test_model_loading PASSED [ 59%]
tests/unit/test_model.py::TestModelInstantiation::test_tokenizer_loading PASSED [ 62%]
tests/unit/test_model.py::TestModelInstantiation::test_model_forward_pass_with_dummy_data PASSED [ 65%]
```



```

tests/unit/test_data_extraction.py::test_encoding_fallback PASSED [ 15%]
tests/unit/test_data_extraction.py::test_parser_error_fallback PASSED [ 18%]
tests/unit/test_data_processing.py::test_clean_text_with_real_data PASSED [ 21%]
tests/unit/test_data_processing.py::test_normalize_reviews_creates_clean_column PASSED [ 25%]
tests/unit/test_data_processing.py::test_normalize_reviews_no_content_column PASSED [ 28%]
tests/unit/test_data_processing.py::test_tokenization_adds_columns PASSED [ 31%]
tests/unit/test_data_processing.py::test_tokenization_padding_and_mask PASSED [ 34%]
tests/unit/test_data_processing.py::test_tokenization_missing_clean_content PASSED [ 37%]
tests/unit/test_inference.py::test_model_load_and_predict PASSED [ 40%]
tests/unit/test_inference.py::test_positive_sentiment PASSED [ 43%]
tests/unit/test_inference.py::test_negative_sentiment PASSED [ 46%]
tests/unit/test_inference.py::test_neutral_sentiment FAILED [ 50%]
tests/unit/test_inference.py::test_empty_input PASSED [ 53%]
tests/unit/test_inference.py::test_prediction_consistency PASSED [ 56%]
tests/unit/test_model.py::TestModelInstantiation::test_model_loading PASSED [ 59%]
tests/unit/test_model.py::TestModelInstantiation::test_tokenizer_loading PASSED [ 62%]
tests/unit/test_model.py::TestModelInstantiation::test_model_forward_pass_with_dummy_data PASSED [ 65%]
tests/unit/test_model.py::TestModelInstantiation::test_model_output_logits_range PASSED [ 68%]
tests/unit/test_model.py::TestModelInstantiation::test_model_predictions_sum_to_one PASSED [ 71%]
tests/unit/test_model.py::TestComputeMetrics::test_compute_metrics_perfect_predictions PASSED [ 75%]
tests/unit/test_model.py::TestComputeMetrics::test_compute_metrics_all_wrong PASSED [ 78%]
tests/unit/test_model.py::TestComputeMetrics::test_compute_metrics_returns_correct_keys PASSED [ 81%]
tests/unit/test_model.py::TestPrepareDatasets::test_prepare_datasets_structure PASSED [ 84%]
tests/unit/test_model.py::TestModelWithRealTokenizer::test_model_with_tokenized_text PASSED [ 87%]
tests/unit/test_model.py::TestDummyBatchInference::test_run_dummy_batch_inference PASSED [ 90%]
tests/unit/test_model.py::TestDummyBatchInference::test_compare_predictions_to_expected_shape PASSED [ 93%]
tests/unit/test_model.py::TestDummyBatchInference::test_evaluate_accuracy_and_f1_score PASSED [ 96%]
tests/unit/test_model.py::TestDummyBatchInference::test_batch_inference_consistency PASSED [100%]

```

3. Dependency & Version Issues

- Several library mismatches (especially with `transformers` and `torch`) caused runtime errors on different machines.
- This was solved by syncing the `requirements.txt` and enforcing a consistent environment using Python 3.12.

6. Future Improvements

If given more time, the following enhancements would further improve the project:

- 1. Train with a Larger Dataset**
Improve generalization and accuracy by using a more diverse, large-scale dataset (e.g., IMDb or Yelp reviews).
- 2. Reduce Training Time**
 - Use smaller, optimized transformer architectures.
 - Explore gradient accumulation or mixed-precision training.
- 3. Fix the Failed Test with Retraining**
 - Fully retrain the model for all epochs to ensure inference outputs match expected shapes.
 - Validate using a clean test split.

4. **Improve Deployment**

- Wrap the inference script into a **Flask** or **FastAPI** endpoint for real-time predictions.
- Add Docker support for reproducibility.

5. **Enhanced CI/CD Pipeline**

Automate model retraining and deployment using **GitHub Actions** or **MLflow**.

6. **User Interface**

Create a minimal web app allowing users to type text and get live sentiment predictions.

7. Conclusion

This project successfully demonstrates the principles of **MLOps**, teamwork, and reproducible AI development.

Despite facing runtime and training limitations, our modular structure, testing pipeline, and documentation reflect solid engineering practices.

Each member contributed effectively within their role, ensuring that the resulting pipeline — even with its constraints — remains functional, transparent, and extensible for future improvements.